axiateam
RDT company

Módulo 2

Entorno de Desarrollo

Curso: TypeScript

60 min

Objetivos de Aprendizaje

- Configurar un **entorno de desarrollo profesional** para TypeScript
- Dominar las **herramientas de VS Code** para TypeScript
- Utilizar **Git** para el control de versiones del proyecto
- Gestionar **dependencias con npm**

1. VS Code

El editor profesional

Por qué VS Code?

Ventajas

- Gratis y open source
- Multiplataforma: Windows, macOS, Linux
- IntelliSense: autocompletado inteligente
- Extensiones: ecosistema huge

Características

- Terminal integrada
- Integración con Git
- Depuración integrada
- Live Server

Extensiones esenciales para TypeScript

Extensión	Descripción	Enlace
TypeScript Nightly	Soporte de la última versión	Marketplace
ESLint	Detección de errores y estilo	Marketplace
Prettier	Formateo automático de código	Marketplace
Error Lens	Errores visibles en el editor	Marketplace
Path IntelliSense	Autocompletado de rutas	Marketplace

Atajos esenciales

Atajo	Acción
`Ctrl+Shift+P`	Paleta de comandos
`F12`	Ir a definición
`Ctrl+Click`	Navegar a definición
`Ctrl+Space`	Sugerencias de tipo

Atajo	Acción
`Ctrl+Shift+O`	Ir a símbolo
`Ctrl+` `	Mostrar terminal
`Ctrl+Shift+E`	Explorador
`Ctrl+Shift+F`	Buscar en archivos

Configuración recomendada

```
{
  "editor.formatOnSave": true,
  "editor.defaultFormatter": "esbenp.prettier-vscode",
  "editor.codeActionsOnSave": {
    "source.fixAll.eslint": "explicit"
  },
  "typescript.preferences.includePackageJsonAutoImports": "on",
  "files.autoSave": "afterDelay",
  "eslint.validate": ["typescript", "typescriptreact"]
}
```

Abre settings.json con `Ctrl+Shift+P` > "Open Settings (JSON)"

2. Instalación en Linux

Configuración completa

Instalar VS Code

Debian/Ubuntu

```
sudo apt update
sudo apt install wget gpg
wget -qO- https://packages.microsoft.com/keys/microsoft.asc | gpg --dearmor > packages.microsoft.gpg
sudo install -D -o root -g root -m 644 packages.microsoft.gpg /etc/apt/keyrings/packages.microsoft.gpg
sudo sh -c 'echo "deb [arch=amd64,arm64,armhf signed-by=/etc/apt/keyrings/packages.microsoft.gpg] https://packages.microsoft.com/repos/code stable ma
sudo apt update
sudo apt install code
```

3. Git para el Proyecto

Control de versiones

Comandos básicos

```
# Inicializar repositorio
git init

# Ver estado
git status

# Stage todos los archivos
git add .

# Commit con mensaje
git commit -m "feat: primer commit"

# Ver historial
git log
```

Flujo de trabajo con ramas

- `main`: Código estable
- `develop`: Desarrollo
- `feature/nombre`: Nueva funcionalidad
- `git checkout -b feature/nueva-rama`

.gitignore esencial

```
# Node.js
node_modules/
dist/
npm-debug.log*

# TypeScript
*.tsbuildinfo

# Entorno
.env
.env.local

# IDE
.vscode/
.idea/
*.swp
*.swo

# OS
```

Crea `.gitignore` desde el inicio del proyecto

3. Git para el Proyecto

Control de versiones

Comandos básicos

```
# Inicializar repositorio
git init

# Ver estado
git status

# Stage todos los archivos
git add .

# Commit con mensaje
git commit -m "feat: primer commit"

# Ver historial
git log
```

Flujo de trabajo con ramas

- `main`: Código estable
- `develop`: Desarrollo
- `feature/nombre`: Nueva funcionalidad
- `git checkout -b feature/nueva-rama`

.gitignore esencial

```
# Node.js
node_modules/
dist/
npm-debug.log*

# TypeScript
*.tsbuildinfo

# Entorno
.env
.env.local

# IDE
.vscode/
.idea/
*.swp
*.swo

# OS
```

Crea `.gitignore` desde el inicio del proyecto

4. npm y Dependencias

Gestión de paquetes

package.json

```
{
  "name": "mi-proyecto-typescript",
  "version": "1.0.0",
  "description": "Mi primer proyecto TypeScript",
  "scripts": {
    "build": "tsc",
    "dev": "tsc --watch"
  },
  "devDependencies": {
    "typescript": "^5.3.0"
  }
}
```

Comandos npm esenciales

- `npm init -y` - Inicializar proyecto
- `npm install paquete` - Dependencia de producción
- `npm install -D paquete` - Dependencia de desarrollo
- `npm run build` - Ejecutar script de build

5. Primera Prueba de Funcionamiento

Verificar el entorno

```
# 1. Verificar Node.js
node --version

# 2. Verificar npm
npm --version

# 3. Verificar Git
git --version

# 4. Verificar TypeScript
tsc --version

# 5. Verificar VS Code
code --version
```

Crear primer proyecto TypeScript (Caso 1)

```
# 1. Crear carpeta del proyecto
mkdir task-manager
cd task-manager

# 2. Inicializar npm
npm init -y

# 3. Instalar TypeScript
npm install -D typescript

# 4. Inicializar tsconfig
npx tsc --init

# 5. Crear archivo TypeScript
echo 'const greeting: string = "Hola TypeScript";' > index.ts

# 6. Compilar
npx tsc
```

¡Felicidades! Has creado tu primer proyecto TypeScript.

Resumen

Puntos Clave

- **VS Code** es el editor recomendado con IntelliSense nativo para TypeScript
- Instala **extensiones esenciales**: TypeScript Nightly, ESLint, Prettier
- **Git** permite control de versiones y trabajo colaborativo
- **npm** gestiona las dependencias del proyecto
- El **flujo básico**: npm init → npm install → tsc → node

Referencias

Recursos Adicionales

- VS Code - IntelliSense
- Git - Documentación oficial
- npm - Documentación
- TypeScript - Documentación oficial

Gracias!

Módulo 2 Completado

Siguiente: Módulo 3 - Compilación a JavaScript